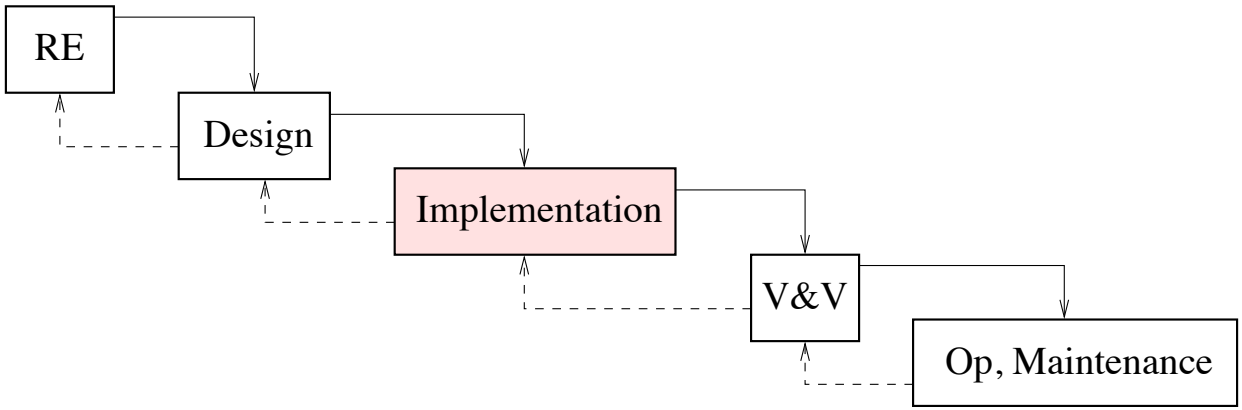
Secure Coding

Security Engineering

Srđan Krstić
ETH Zürich

Context: implementation



Even with a great design, an implementation may be insecure

1. The implementation deviates from the design.

2. The design leaves many options open, including insecure ones.

   E.g., lengths of passwords, their storage, etc.

3. Concretization may introduce new vulnerabilities.

   E.g., buffer overflows, race conditions, etc.

We will focus on implementation-specific vulnerabilities.

Why study vulnerabilities?

- Vital to understanding what can go wrong in **implementation**
  - What kinds of faults open the door to attacks?
  - Failure to understand underlying causes leads to insecure code, partial solutions, etc.
- Essential for **risk analysis**
- Helps focus efforts during **V&V**
- Warning: studying is good, but exploitation is criminal

  $\Longrightarrow$ Experiment only in a controlled environment.

Organization in two parts

- **Part I: Stand-alone applications**
  - Buffer overflows
  - Format string vulnerabilities
  - Names, links, and race conditions
- **Part II: Web-based applications**
  - Overview
  - SQL injections
  - Cross-site scripting
  - Authentication and session management

Coverage objective: explain different practically relevant, representative examples. Also how to **think** about vulnerabilities.

Road map: buffer overflows

☞ **Motivation**

- Pointers in C
- Memory layout
- Buffer overflows
- Defense
- Summary

Buffer overflows

- Public enemy #1 for over 30 years

  Various statistics suggest 15%–70% of all vulnerabilities are memory safety issues, with buffer overflows a primary problem.

- **Example:** Morris Internet Worm (November 1988)
  - Attacked Sun3 and VAX systems running BSD Unix.
  - Cleverly built, e.g., named itself "sh" (hard to detect) and set maximum core-dump size to 0 bytes (hard to catch).
  - Propagated itself using various exploits, including a buffer overflow attack against the finger daemon *fingerd*.
  - Did not delete files, but caused financial loss between $100,000 and $1,000,000 (US General Accounting Office).

A typical example

Mozilla Foundation Security Advisory 2009-56

Title: Heap buffer overflow in GIF color map parser
Impact: Critical
Announced: October 27, 2009
Products: Firefox, SeaMonkey
Fixed in: Firefox 3.5.4, Firefox 3.0.15, SeaMonkey 2.0

DESCRIPTION

Security research firm iDefense discovered a heap-based buffer overflow in Mozilla's GIF image parser. This vulnerability can be used by an attacker to crash a victim's browser and run arbitrary code on their computer.

Where do buffer overflows occur?
Almost everywhere!

- Applications, local or available over the web

  Web application overflows provide an entry point for attackers

- Browsers and their plugins (e.g., last example)

  Just clicking on a link suffices to compromise the client

- Operating systems and protocol stacks

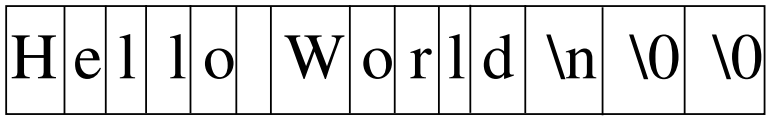
  Affecting servers, PCs, smart phones, industrial control systems

- Implementations of cryptographic algorithms

  Example: several of the algorithms submitted to NIST for the SHA-3 standard had buffer overflows

- Firewalls, security appliances, type-safe language interpreters

Buffer overflows



- A **buffer** is a contiguous region of memory storing data of the same type, e.g., characters.
- A **buffer overflow** occurs when data is written past buffer's end.
- The resulting damage depends on:
  - Where the data spills over to
  - How this memory region is used (e.g., flags for access control)
  - What modifications are made
- **Example:** Morris attack on finger
  - Intended use: **finger basin@inf.ethz.ch**
  - Abuse: **finger** ⟨**exploit-code**⟩ . . . ⟨**return-address**⟩
  - Overwrites the return address and provides exploit code

Road map: buffer overflows

- Motivation

☞ **Pointers in C**

- Memory layout
- Buffer overflows
- Defense
- Summary

C primer

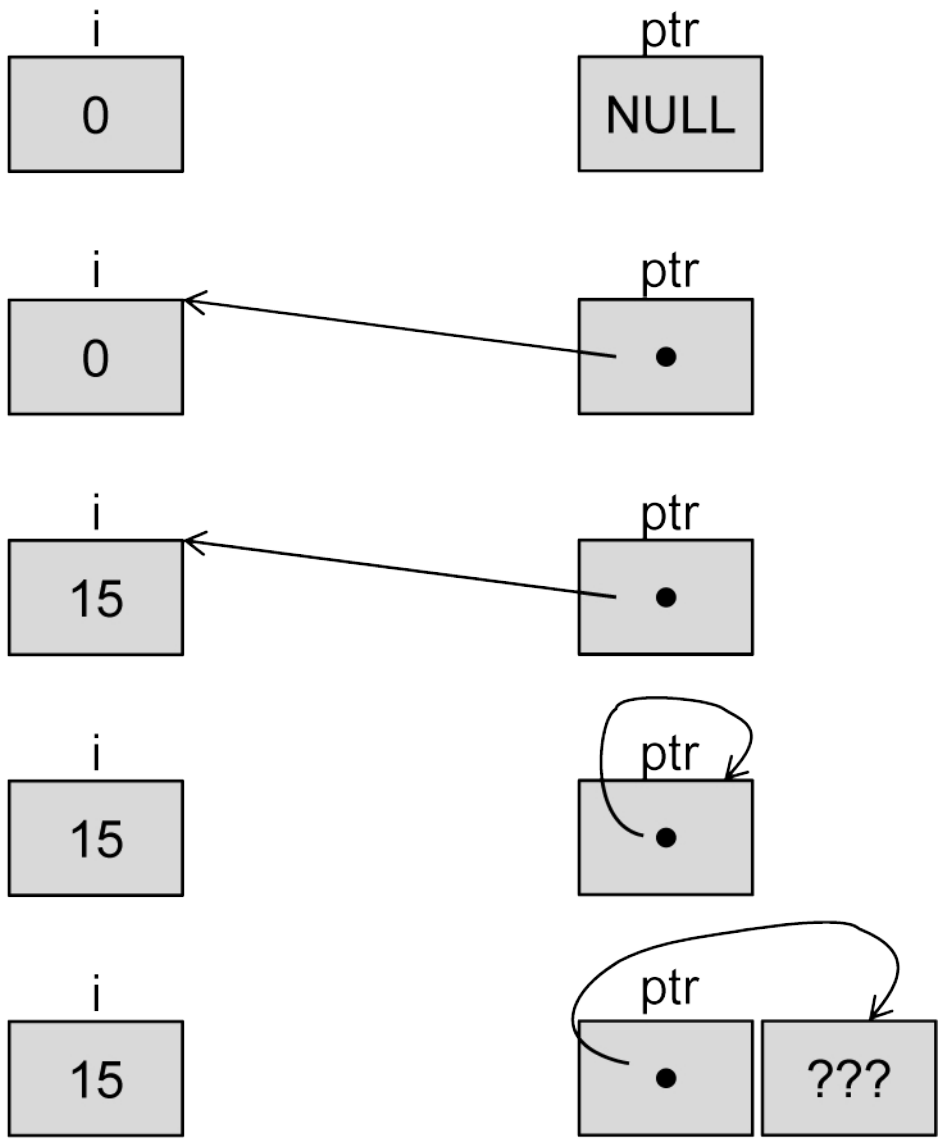
- C(++) is powerful, widespread, and undisciplined!
- Basic data types

  Char (1 byte), int ($\geq$ 2 bytes), long ($\geq$ 4 bytes) ...

- Pointers: expressions like **int *ptr**
  - **ptr** is the **address** of a memory cell
  - ***ptr** is the **contents** of the memory cell
- Address taken using **&**. For **i** an integer and **f** a function,
  - **&i** is the **address** of the cell storing **i**
  - **&f** is the **address** of a function **f**

Pointers — examples

```
int i = 0;
int *ptr = NULL;


ptr = &i;


*ptr = 15;


ptr = &ptr;


ptr++;
```



Arrays

- Declaring and accessing an array
  - **char buf[8];** declares a buffer for up to 8 characters
  - Elements are **buf[0]**, **buf[1]**, .., **buf[7]**
  - No bounds checking, so **buf[13]** is also possible
- Arrays are similar to pointers: store address where buffer begins
- Some equalities

$$\&(buf[0]) = buf \qquad buf[0] = *(buf+0)$$
$$\&(buf[1]) = buf + 1 \qquad buf[1] = *(buf+1)$$

- Strings are arrays of characters, with **'\0'** at end
  - E.g., string **"hi"** represented by **'h' 'i' '\0'**
  - No explicit information about strings' length
  - This savings is part of the problem!

Input, output and string handling functions

- **getchar()** reads a character from standard input
- **putchar(c)** writes a character to standard output
- **printf("Page %i has title %s\n", pageno, title)**
  - String printed may have place-holders, replaced by arguments
  - Place holders provide formatting instructions.
    E.g., **%i** stands for an integer in decimal and **%s** for a string
- Various other functions store strings in buffers. E.g.
  - **gets(dst)** read string from **stdin** into **dst**
  - **strcpy(dst,src)** copy string **src** into buffer **dst**
  - **sprintf(dst, fmt-str,exp)** print string into buffer **dst**
  - **scanf**, **fscanf**, ...

A vulnerable program

buffer of limited size

```
#include <stdio.h>
int main()
  { char buf[8]                          //buffer for storing the input string
    char ch;                             // auxiliary variable
    char *ptr = buf;                     // auxiliary pointer
    while ((ch=getchar()) != '\n'        // terminate on EOL
          && ch != -1)                   // terminate on error
       {  *ptr = ch;                     // store character
          ptr++;                         // increment pointer
       }
    *ptr = '\0';                         // terminate the string
    printf("%s\n",buf);
    return 0;
  }
```

is written without limits

Example executions:

```
$ ./my-getstring
example
example
```

```
$ ./my-getstring
long example
long example
illegal instruction
```

```
$ ./my-getstring
very long example
very long example
segmentation fault
```

Road map: buffer overflows

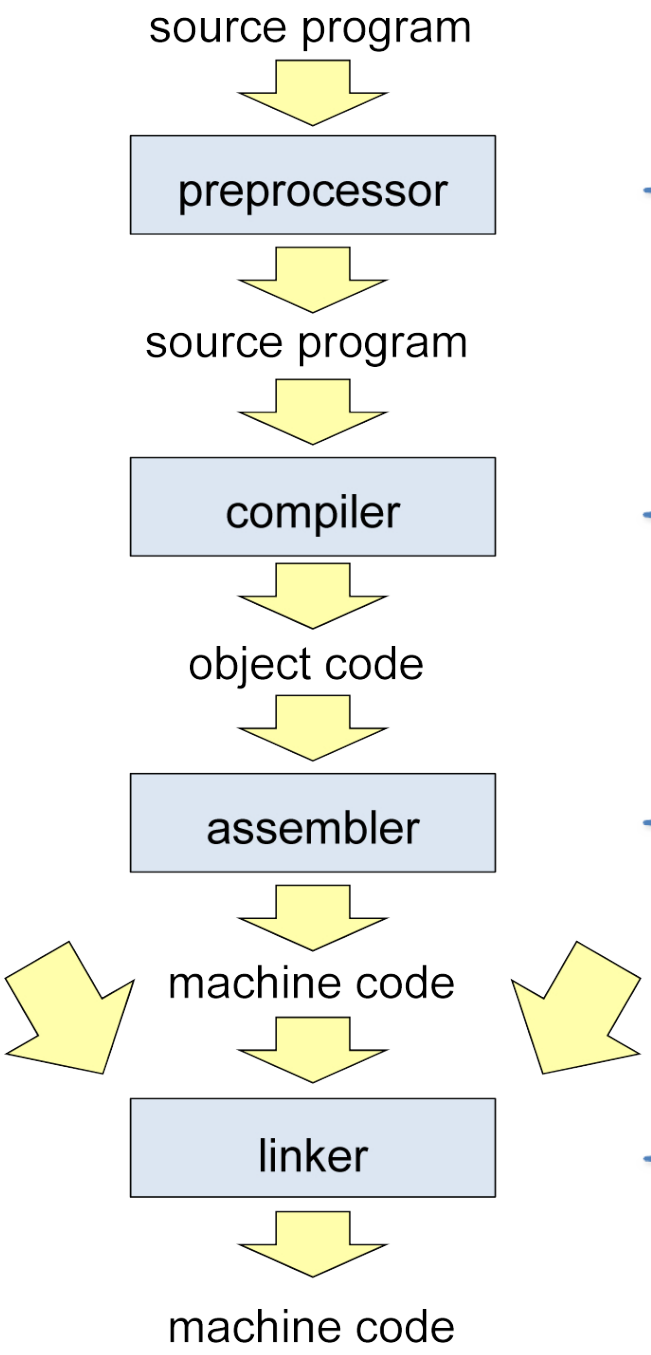
- Motivation
- Pointers in C

☞ **Memory layout**

- Buffer overflows
- Defense
- Summary

What follows is for Linux on Intel x86 family!

Compilation



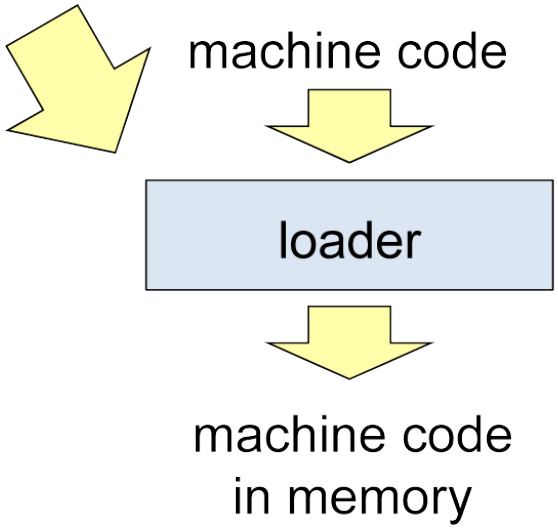
- Expand macros
- Include header files

- Lexical analysis
- Syntactic analysis
- Semantic analysis
- Optimization
- Code generation

- Translate object code into machine code

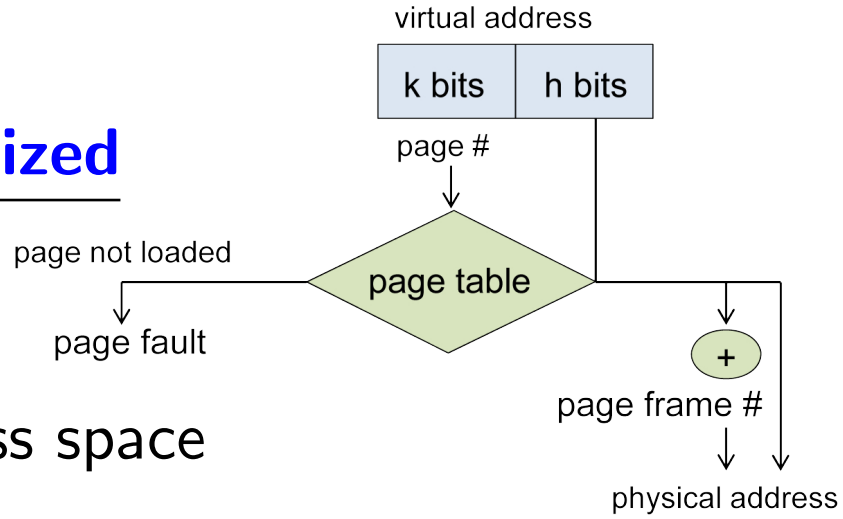
- Combine multiple modules to one program
  - Arrange modules in memory
- Resolve external references
  - Each module says what it needs and what it provides

Loading



- Loading of program into the address space of the process
- Adapting addresses or initializing a relocation register (hardware supported relocation)
- Dynamic loading and linking of libraries for unresolved references

Recall how (virtual) memory is organized



- Memory named by virtual addresses
- Each process has its own virtual address space
- Protection: process can only access memory in its own space

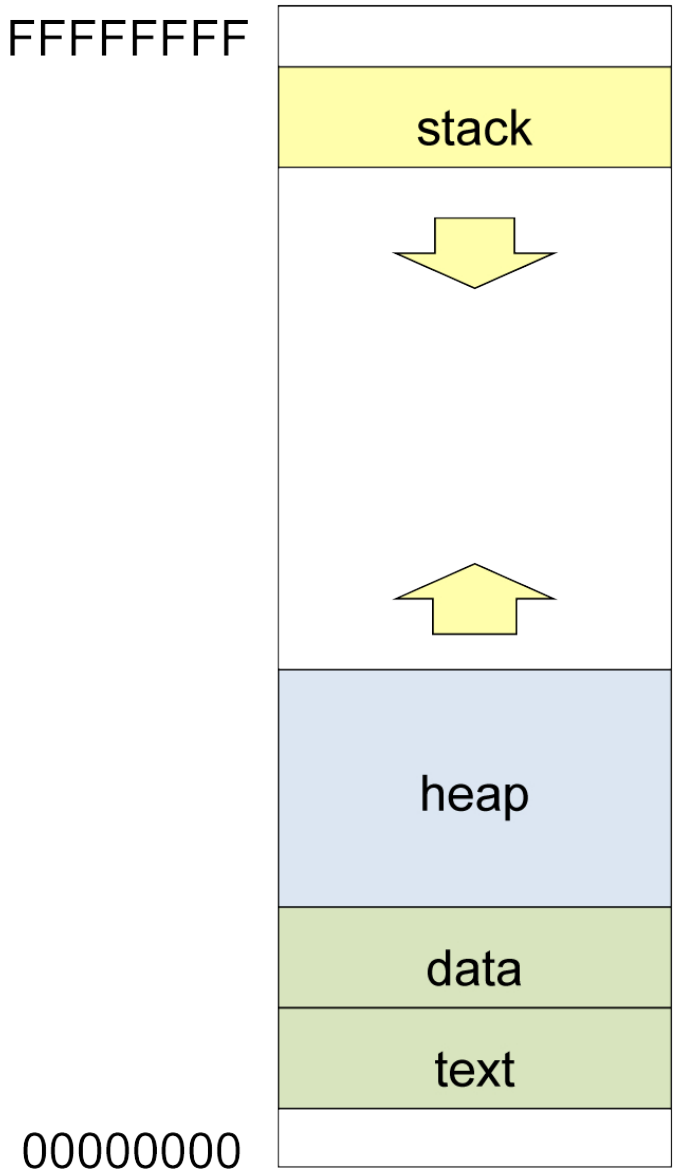
A program in memory

```
$ gdb my-getstring
(gdb) disas main
Dump of assembler code for function main:

0x8048440  push   %ebp
0x8048441  mov    %esp, %ebp
0x8048443  sub    $0x18,%esp
0x8048446  lea    0xfffffff8(%ebp),%eax
0x8048449  mov    %eax, 0xfffffff0(%ebp)
0x804844c  call   0x80482e8 <getchar>
0x8048451  mov    %eax, %eax
0x8048453  mov    %al,0xfffffff7(%ebp)
0x8048456  mov    0xfffffff7(%ebp),%al
0x8048459  cmp    $0xa,%al
0x804845b  je     0x8048478 <main+56>
0x804845d  cmpb   $0xff,0xfffffff7(%ebp)
0x8048461  jne    0x8048468, <main+40>
0x8048463  jmp    0x8048478, <main+56>
0x8048465  lea    0x0(%esi), %esi
0x8048468  mov    0xfffffff0(%ebp),%edx
0x804846b  mov    0xfffffff7(%ebp),%al
0x804846e  mov    %al,(%edx)
0x8048470  lea    0xfffffff0(%ebp), %eax
0x8048473  incl   (%eax)
0x8048475  jmp    0x804844c <main+12>
0x8048477  nop
0x8048478  mov    0xfffffff0(%ebp),%eax
0x804847b  movb   $0x0,(%eax)
0x804847e  sub    $0x8,%esp
0x8048481  lea    0xfffffff8(%ebp),%eax
0x8048484  push   %eax
0x8048485  push   $0x8048508
0x804848a  call   0x8048328 <printf>
0x804848f  add    $0x10,%esp
0x8048492  mov    $0x0,%eax
0x8048497  leave
0x8048498  ret
0x8048499  lea    0x0(%esi),%esi
0x804849c  nop
0x804849d  nop
0x804849e  nop
0x804849f  nop
```

Layout of virtual memory

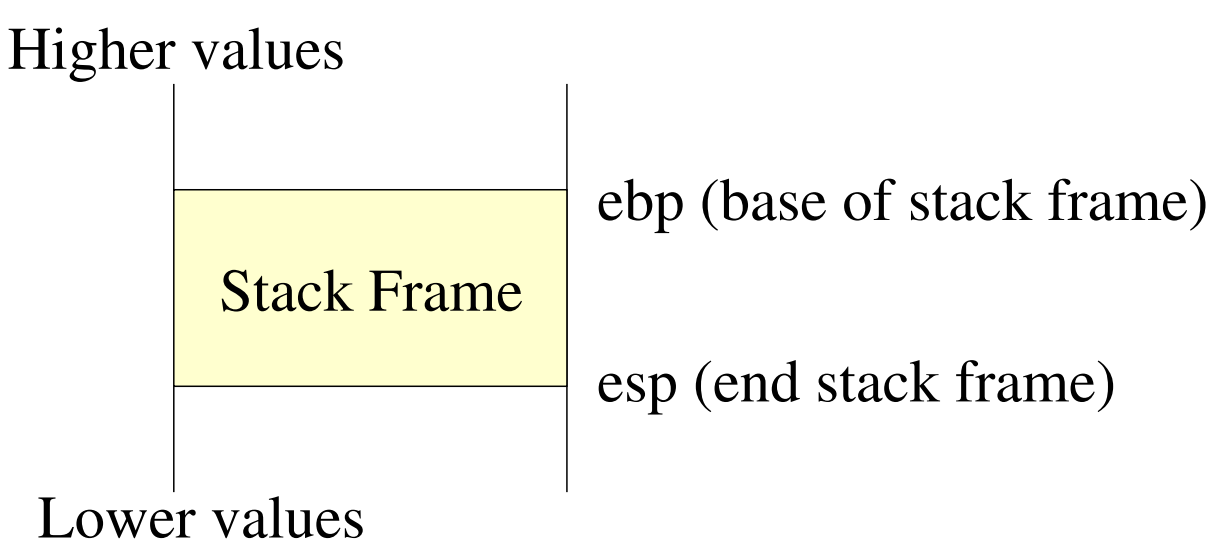
Linux on Intel x86 family

- Stack grows downward
- Stack frame holds
  - Calling parameters
  - Local variables for functions
  - Various addresses
- Heap grows upwards
  - Dynamically allocated storage generated using alloc or malloc
- Data: statically allocated storage
- Text: Executable code, read only



Structure of the stack

- Stack grows downwards: one **stack frame** per subroutine called



- Hardware registers include:

  ebp (extended) base pointer: start current frame
  esp (extended) stack pointer: top (end) of stack

Another vulnerable program

```
int main (void) {
   char pw[8];
   sprintf (pw, "root-pw");
   if (authenticate(pw) > 0) {
     printf ("[root]$ …\n");
   } else {
     printf ("Root: incorrect password\n");
   }
   return 0;
}
```

```
#include <stdio.h>
int authenticate (char* pw) {
   int result = 0;
   char buf[8];
   printf ("Root Password: ");

   if (gets(buf) !=0 ){
     if (strcmp (buf, pw) == 0) {
       result = 1;
     }
   }
   return result;
}
```

- $ **my-authenticate**

  Root Password: guess

  Root: incorrect password

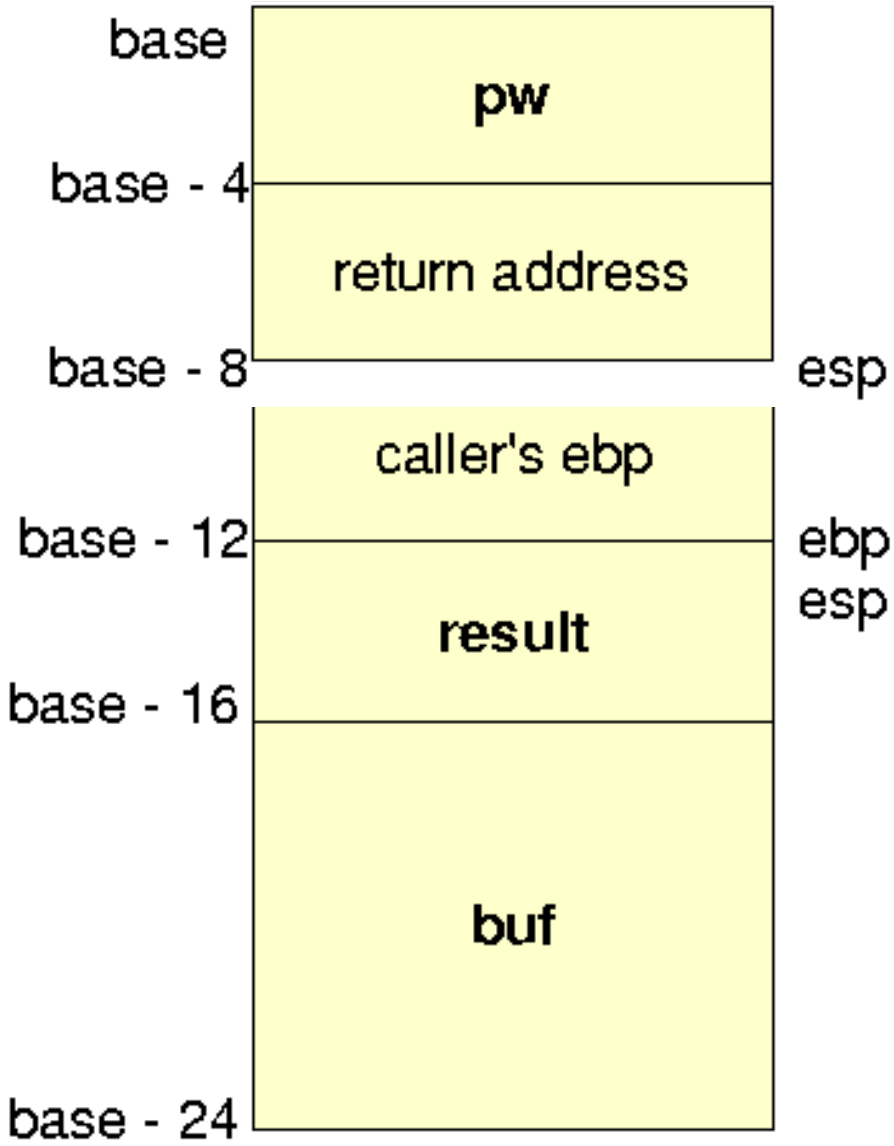
- $ **my-authenticate**

  Root Password: root-pw

  [root]$ ...

Stack dynamics in example

1. start (of main)
2. push argument **pw** onto stack
3. call **authenticate** (pushes return address onto stack)
4. push caller's **ebp** onto stack
5. copy esp into **ebp**
6. decrement **esp**, creating space for local vars
7. Compute **authenticate** function (details omitted)
8. copy caller's **ebp** into **esp**
9. pop vars and caller's **ebp** from stack
10. return



Road map: buffer overflows

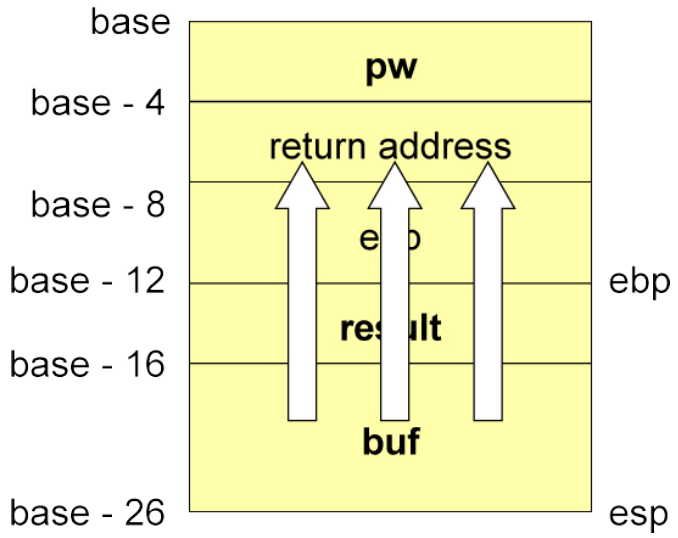
- Motivation
- Pointers in C
- Memory layout
- ☞ **Buffer overflows**
- Defense
- Summary

Where is vulnerability in authenticate?

- Some non-problems
  - Variables **result** and **buf** are initialized prior to use.
  - Programming logic is OK. **result** is 1 iff input equals password supplied by **pw**.
- The problem: **gets** implementation allows the overflow of **buf**.

```
#include <stdio.h>
int authenticate (char* pw) {
   int result = 0;
   char buf[8];
   printf ("Root Password: ");

   if (gets(buf) !=0 ){
     if (strcmp (buf, pw) == 0) {
       result = 1;
     }
   }
   return result;
}
```

Example: a long password



- In a buffer overflow, data from **buf** spills over into **result**.
- Can change value of **result**.

  result initialized to 0 and updated only if authentication succeeds.
- Works (conceptually) as follows

  $ my-authenticate
  Root Password: aaaaaaaa\x01\x00
  [root]$ ...

  buf filled with **"aaaaaaaa"** and hence **'\x01'** overwrites **result**.
- **Exploit violates data integrity!**

Understanding C is not enough!

- Last example showed violation of data integrity.

  Moreover, variables modified without explicit assignment!

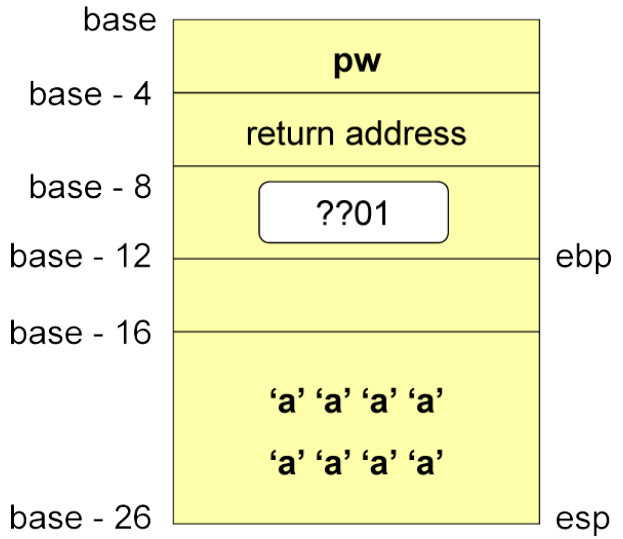
- Recall: C language offers standard abstractions.
  - Memory is associated with variables.
  - Memory is updated by assignments to variables.
  - Control flow is described by **if**, **for**, etc.

- Buffer overflows violate these abstractions.
  Results are security vulnerabilities.

- **Incredible but true:**
  Thinking purely on the level of C misses these possibilities!

Long password (cont.)

- In reality, buffer overflows are a bit trickier.
- There could be a gap between **buf** and **result**.
- If input is too long then **ebp** or **return** may be effected.



- Entering hexadecimal values **'\x01'** and **'\x00'**.

  Here one can enter input strings using a program via a pipe.

Can we program defensively against such errors?

A defense: restoring variables

```
#include <stdio.h>
int authenticate (char* pw) {
   int result = 0;
   char buf[8];
   printf ("Root Password: ");

   if (gets(buf) !=0 ){
     if (strcmp (buf, pw) == 0) {
       result = 1;
     }
   }
   return result;
}
```

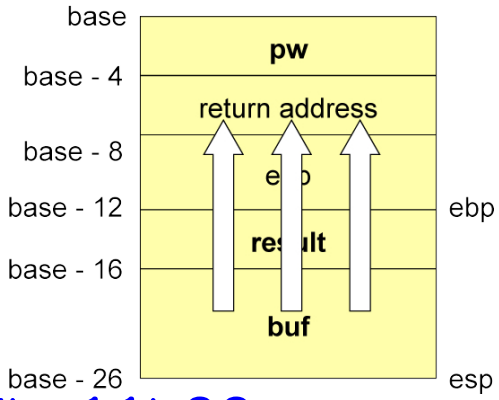
```
#include <stdio.h>
int authenticate (char* pw) {
   int result = 0;
   char buf[8];
   printf ("Root Password: ");

   if (gets(buf) !=0 ){
     if (strcmp (buf, pw) == 0) {
       result = 1;
   } else { result = 0; }
   }
   return result;
}
```

- Result is updated after **gets**.

  Now result is correct even if **buf** overflows into **result**.

- Would you recommend this defense?

Example: a very long password

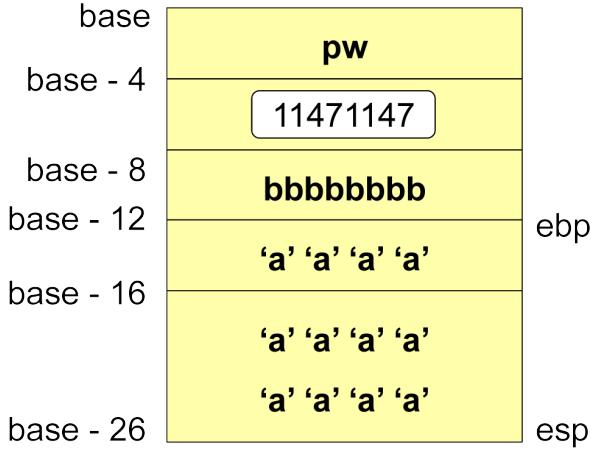


- What happens on:

  $ my-authenticate
  Root Password:
  aaaaaaaaaaaa\xbb\xbb\xbb\xbb\x47\x11\x47\x11\00

- Overwrites **result**, **ebp**, and **return**

  buf: "aaaaaaaa"
  result: "aaaa"
  stored ebp: bb bb bb bb
  return address: 11 47 11 47



- Call of **ret** (when leaving subroutine) pops 11 47 11 47 off stack, starting execution at this memory location.

Overflow has destroyed integrity of code-flow!

Exploit code (or where evil doers jump)

- Where would a malicious attacker jump to?

  Common target: code that creates a (root-)shell.

- Where in memory does this code go?
  - Exploit code typically placed on the stack.
  - Usually, within the very buffer that is overflowed.

- Return address must point exactly to the exploit's entry point.
  - Non-trivial in practice.
  - Trick used of starting exploit code with a "landing zone" of values representing No-op instructions.

Exploit code (cont.)

- In above approach, exploit code and landing zone fit into buffer.
- Alternatively, attacker places exploit code:
  - **On the stack:** into parameters or other local variables.
  - **On the heap:** into some dynamically allocated memory region.
  - **Into environment variables** (on stack).
- Another alternative is to abuse existing code.

  E.g., jump to fragments of the program code or library functions.
- Clever abuses have become a kind of hacker sport![1]

[1] Jonathan Pincus, Brandon Baker. *Beyond Stack Smashing: Recent Advances in Exploiting Buffer Overruns*, IEEE Security & Privacy, 2004.

Road map: buffer overflows

- Motivation
- Pointers in C
- Memory layout
- Buffer overflows

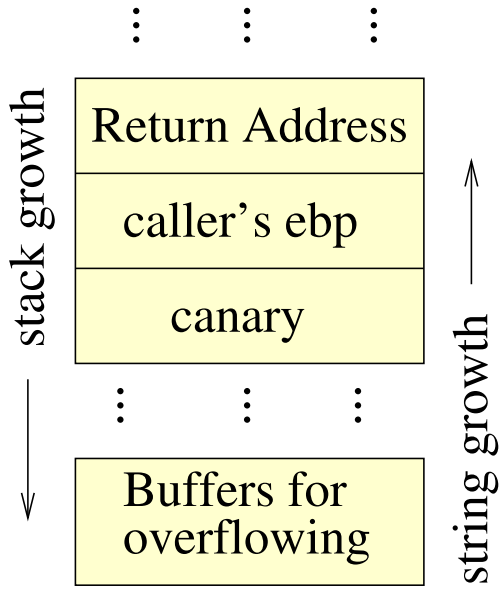
☞ **Defense**

- Summary

Insert a canary

- A **canary** is a value on the stack whose value is tested before returning.

```
void function (...) {
   int canary;
   char buf[MAX_SIZE]; // + other declarations

   canary = CANARY_VALUE;
   gets(buf);    // or another dangerous operation
   ...
   if(canary != CANARY_VALUE)
      exit
   return; }
```



- A canary is a random value (hard for attacker to guess) or a value composed of different string terminators (CR, LF, Null, -1).
- Implementations for GCC and Microsoft Visual C++ compilers. Known attacks exist (see Pincus/Baker).

Automatic array bounds checking

- **Approach:** compiler automatically adds an explicit check to each array access during code generation.

  Example: GCC enhancements of Jones&Kelly.

- Drawbacks
  - It can be difficult to determine the bounds of an array.
  - Loss of performance can be substantial, e.g., factor 10 – 30!
  - Some compilers only check explicit array references, like **buf[n]**, but not pointer references, like ***(buf+n)**.

Defensive programming

- Avoid unsafe library functions, e.g.,strcpy, gets, ...
  - Replace with safe variants, e.g., str**n**cpy, **f**gets, ...
  - E.g., replace **strcpy(dst,src)** with **strncpy(dst,src,dst_size-1)**
- Always check bound of arrays when iterating over them.
- Mechanisms for doing this:
  - Careful programming
  - Audit teams
  - Use grep or more sophisticated tools
- Limitations: error prone and audits are time consuming.

Defensive programming (cont.)

BUGS

Never use gets(). Because it is impossible to tell without knowing the data in advance how many characters gets() will read, and because gets() will continue to store characters past the end of the buffer, it is extremely dangerous to use. It has been used to break computer security. Use fgets() instead.

— From gets(3) manual page

Avoid C (++)

- Use a language that is type safe.
  - Length of an array is part of its type.
  - Assigning contents of a buffer to a smaller buffer is a type error.
  - **Examples:** Pascal, Java, Haskell, ML, or **Rust**.
- Problems and limitations
  - Often the choice of programming language is not yours.
  - Bugs still possible if run-time environment is programmed in an unsafe language.
    Example: several buffer overflows vulnerabilities in implementations of the JAVA virtual machine.
  - C(++) has its advantages, in particular for writing efficient programs "close to the machine".

Avoid buffers on stack

- Avoid declaration of local array variables in functions.
- Instead, use heap storage, e.g., allocate space with **malloc()**.
- As return address is on the stack, it can not be overwritten by a buffer-overflow on the heap.
- Does this solve all our worries? ▮ No!
  - Heap overflows are also a real problem (not covered here)
  - While they have no effect on control-flow integrity, they can violate data integrity.

Non-Executable Buffers

- Mark stack [or heap] as being non-executable.

  $\Longrightarrow$ attacker cannot run exploit stored in buffers on stack [heap].

- Mechanisms (in HW or SW)
  - Track the upper text segment limit (maximal exec. address).
  - Alternatively, tag pages as (non)executable in the page table.

- Problems and limitations
  - Attacker can still execute code in the text segment.
  - Attacker can still violate data integrity.
  - Sometimes too restrictive (e.g., signals and nested functions).

  Unix signal handlers usually execute on the current process stack. This lets the signal handler return to the point that execution was interrupted in the process.

Address Space Layout Randomization

- Hackers and worms exploit that a vulnerable program, which is widely distributed, can be predictably exploited on many systems.
- Problem can be lessened by randomizing memory layout.
  - Location of stack and heap base in main memory
  - Order libraries are loaded (complicates a jump to library code)
  - Even layout within stack frames by compiler
- Does not eliminate overflow problem.

  Lowers chance of a successful exploit by requiring the attacker (or his exploit code) to guess locations of relevant areas.
- Variants used in many systems/compilers (Linux, Mac, Windows)

Road map: buffer overflows

- Motivation
- Pointers in C
- Memory layout
- Buffer overflows
- Defense

☞ **Summary**

Conclusions and lessons learned

- Buffer overflows can alter program's data and control flow.

  One must understand program execution not just at C level, but also assembler/memory-layout.

- Massive problem and has been so for many years!

- Defense includes:
  - Program defensively, using only functions with bounds checking.
  - Carefully weigh pros and cons of programming language used.
    Do advantages of C outweigh its disadvantages?
  - Compiler/hardware support for preventing overflows.
    This is becoming more standard and does help.
    But be aware of the limitations and overhead.

Organization

- **Part I: Stand-alone applications**
  - Buffer overflows
  - ☞ **Format string vulnerabilities**
  - Names, links, and race conditions
- **Part II: Web-based applications**
  - Overview
  - SQL injections
  - Cross-site scripting
  - Authentication and session management

Implementation problems

- Recall buffer overflows.
  - C language offers certain abstractions.
    - Memory associated with variables and updated by assignments
    - Control flow is described by **if**, **for**, etc.
  - Using functions/data outside of their "normal range" can violate these abstractions and result in security vulnerabilities.
- Here we explore other ways abstractions/assumptions can fail.
- Our intent is not to be comprehensive, but rather to provide some insight and sensitivity to the issues involved.

First a little story about phone phreaking

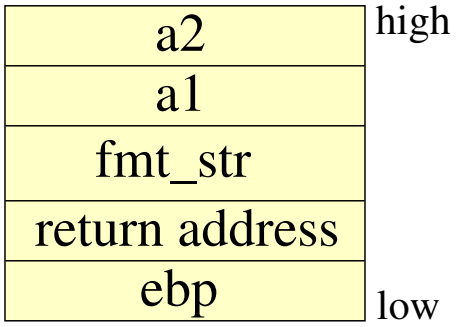
- Arose in 1950s when AT&T introduced automatic, direct-dial, long distance and trunking carriers, which used **in-band signaling**.
- Phreakers (telephone hackers) discovered that whistling a 2600Hz tone could cause a trunk to reset itself. Even better, an appropriate signal sequence allowed free calls!
- What was the problem here?

  The **control channel** and **data channel** were the same.
- Modern day variant: **format string vulnerabilities**, SQL injections, prompt injections, ...

Format string vulnerabilities

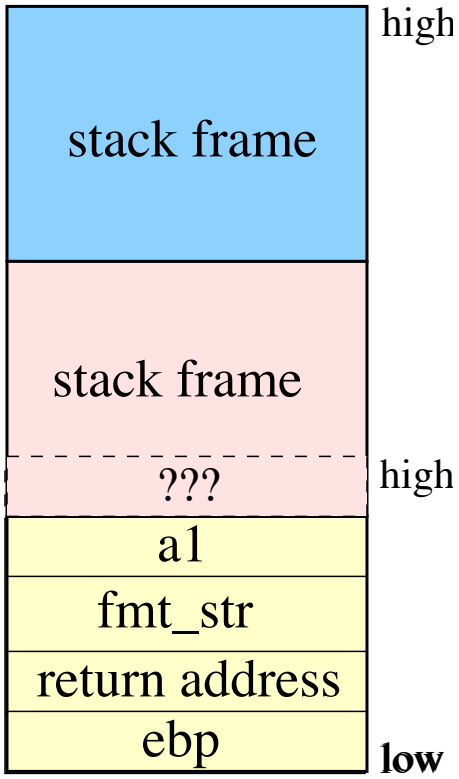
- Format functions have vulnerabilities (problem detected in Y2K).
- Printf syntax: **printf(⟨FORMAT_STRING⟩, ⟨PARAMETER⟩*)**.
- Format string ⟨**FORMAT_STRING**⟩ determines:
  - How the remaining parameters shall be displayed.
    Examples: **%c, %s, %i, %o, %x**
  - How many parameters printf should expect.
- Dangerous when attacker provides format string.
  - He can crash the program.
  - He can read the stack's contents.
  - He can read and overwrite arbitrary memory locations!

Stack frame of a format function



- **Example: printf(fmt_str, a1, a2)**
- **Example**: **printf("%s is assigned %x", name_ptr, val)**

  might print out "loop-counter is assigned 50".



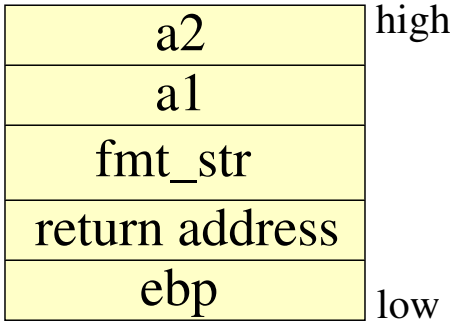
- What happens if fewer arguments are given than format parameters? E.g.,

  printf("%s is assigned %x", name_ptr)

- Forgetting arguments prints stacks contents!

  Question: What will you find there?
  Answer: Stack frame of calling routine.

Vulnerable function calls



- Any difference between **printf("%s", s)** and **printf(s)**?

  The first call is safe while the second is not.

- Danger: The attacker might be able to supply **s**.
- What if ***s** = **"%08x"**? **Answer:** Prints the first stack entry.
- What if ***s** = **"%08x%08x"**? **Answer:** Prints first two entries.
- What about ***s** = **"%s%s%s%s%s"**?
  - Likely to crash the program with a segmentation fault.
  - Stack entries are interpreted as pointers to strings.
  - Segmentation fault if stack entry is not a valid address.

A vulnerable program (1)

```
#include <stdio.h>
int main (int argc, char* argv[]) {
   long *reliable;
   long secret = 0x12345678;

   reliable = (long *) malloc (4);
   *reliable = 0x47114711;
   // format string vulnerability
   if (argc >1) printf(argv[1]);
   printf("\nReliable constant: %x\n", *reliable);
   return 0;
}
```

$ format-string-vuln

Reliable constant: 47114711

$ format-string-vuln hello

hello

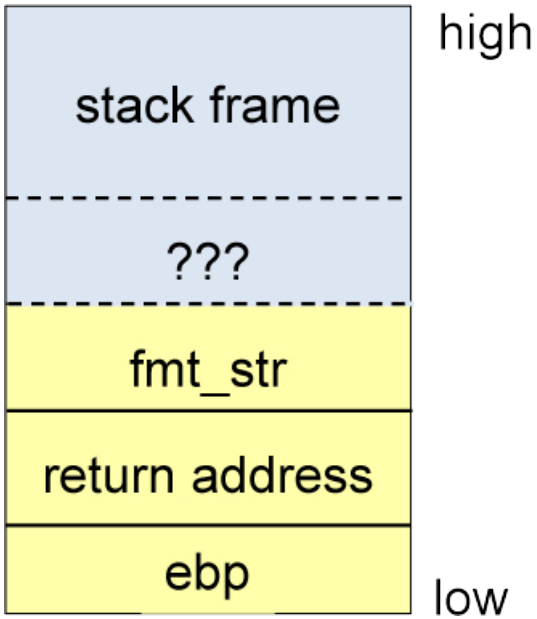
Reliable constant: 47114711

$ format-string-vuln "%s%s%s%s"

Segmentation fault

A vulnerable program (2)

```
#include <stdio.h>
int main (int argc, char* argv[]) {
   long *reliable;
   long secret = 0x12345678;

   reliable = (long *) malloc (4);
   *reliable = 0x47114711;
   // format string vulnerability
   if (argc >1) printf(argv[1]);
   printf("\nReliable constant: %x\n", *reliable);
   return 0;
}
```



$ format-string-vuln "%x%x%x\nThe secret number is: %x"

40013020bfffeaf88048421

The secret number is: 12345678

Reliable constant: 47114711

Here the confidentiality of a secret is leaked!

Modifying contents of memory

- **Note:** **printf** can modify the contents of memory locations.
- **%n** stores the number of characters printed so far in the memory location pointed to by the next argument.
- Example:

  printf("Lucky number:%n", i_ptr); printf("%i\n", *i_ptr);

  - Prints: **"Lucky number: 13"**
  - Reason: **"Lucky number:"** consists of 13 characters.
- Isn't this a nice feature!

A vulnerable program (3)

```
#include <stdio.h>
int main (int argc, char* argv[]) {
   long *reliable;
   long secret = 0x12345678;

   reliable = (long *) malloc (4);
   *reliable = 0x47114711;
   // format string vulnerability
   if (argc >1) printf(argv[1]);
   printf("\nReliable constant: %x\n", *reliable);
   return 0;
}
```

$ **format-string-vuln "%x%x%x%x%n"**
40013020bfffe81880484 2112345678
Reliable constant: 1f

Explanation: String **400...** has 31 = 1f (base 16) characters.

Integrity of data is violated!

Format vulnerabilities

- **Even more is possible**
  - Attacker can read and write arbitrary memory locations.
  - In essence, he owns the program! See references for details.[2]
- **Solution #1**: Programmers must be aware of how library functions work within their operating environment.
  - Similar observations hold for SQL interpreters, PERL, ...
  - Is this a good solution? A practical one?
- **Solution #2**: Provide analysis tools to detect such vulnerabilities.

  E.g., see Shankar et. al. paper cited below.
- **Solution #3**: use weaker library functions.

[2] Umesh Shankar, Kunal Talwar, Jeffrey S. Foster, and David Wagner, *Detecting Format-String Vulnerabilities with Type Qualifiers*, 10th USENIX Security Symposium, 2001. See also http://seclists.org/bugtraq/2000/Sep/214.

Organization

- **Part I: Stand-alone applications**
  - Buffer overflows
  - Format string vulnerabilities
  - ☞ **Names, links, and race conditions**
- **Part II: Web-based applications**
  - Overview
  - SQL injections
  - Cross-site scripting
  - Authentication and session management

We will first review naming, in the context of Unix.

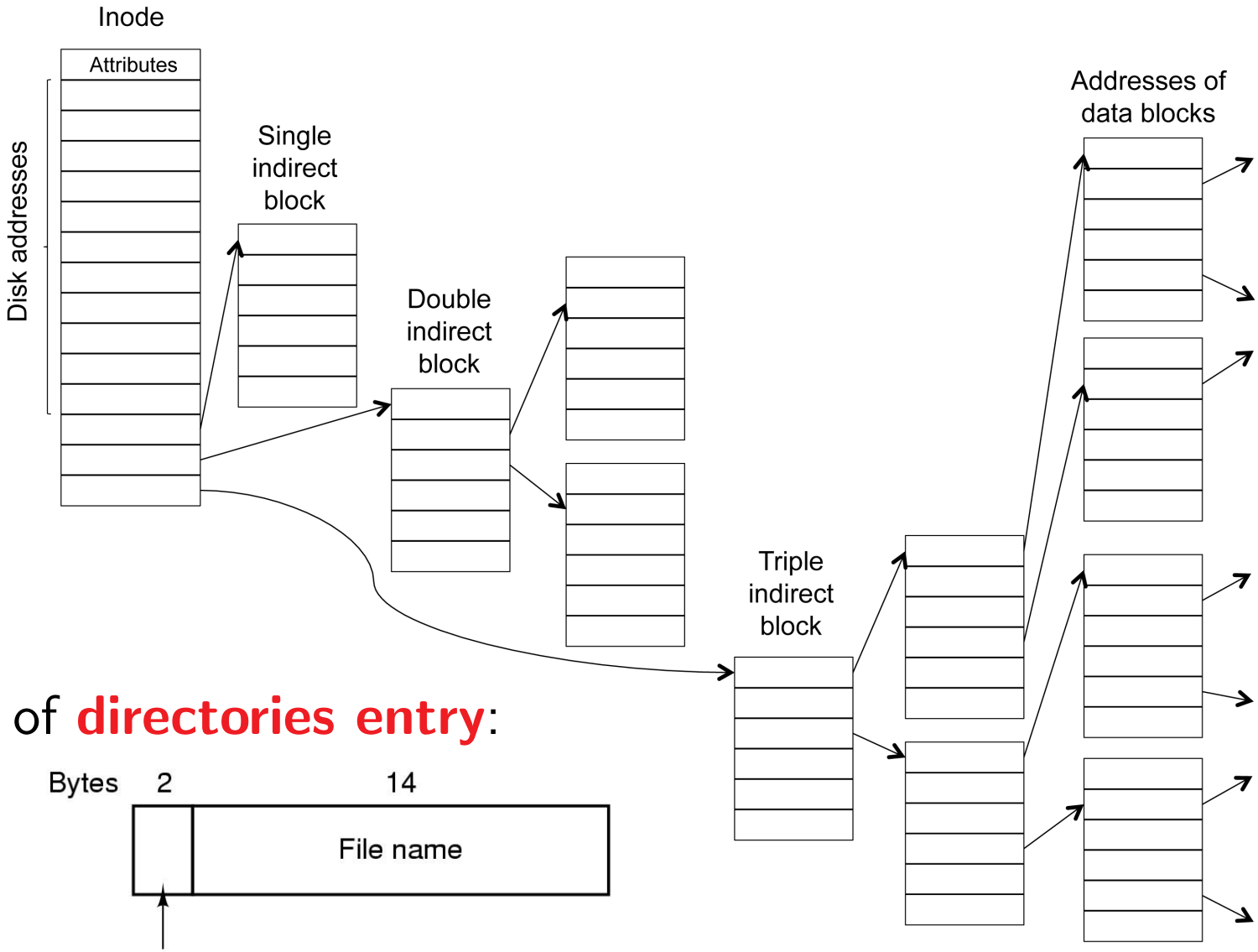
Unix file system

- Following description at an abstract level (roughly Unix V7).
- Directories are hierarchically structured.
  - Contents: directories and (data) files.
  - Root of directory tree is the **root directory /**.
- Users have an associated **current working directory**.
- Directories can be named by

  Absolute file names: e.g., **/usr/bin/ssh**
  Relative (to CWD) file names: e.g., **d1/d2/f**
  Special directory names: **.** (self) and **..** (parent)

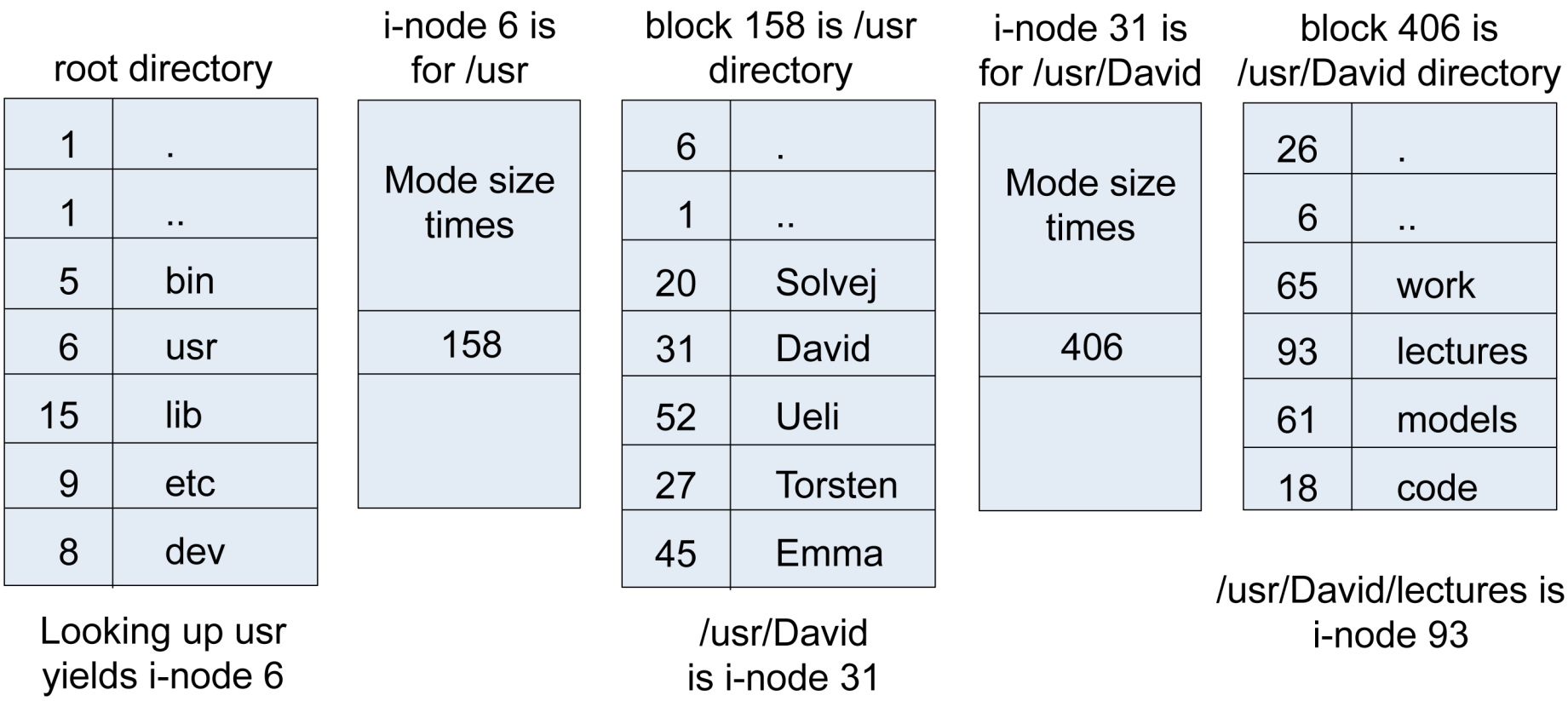
Inodes and directory entries

Each file and directory has an associated **inode** data structure.



Structure of **directories entry**:

Example of file lookup: /usr/David/lectures

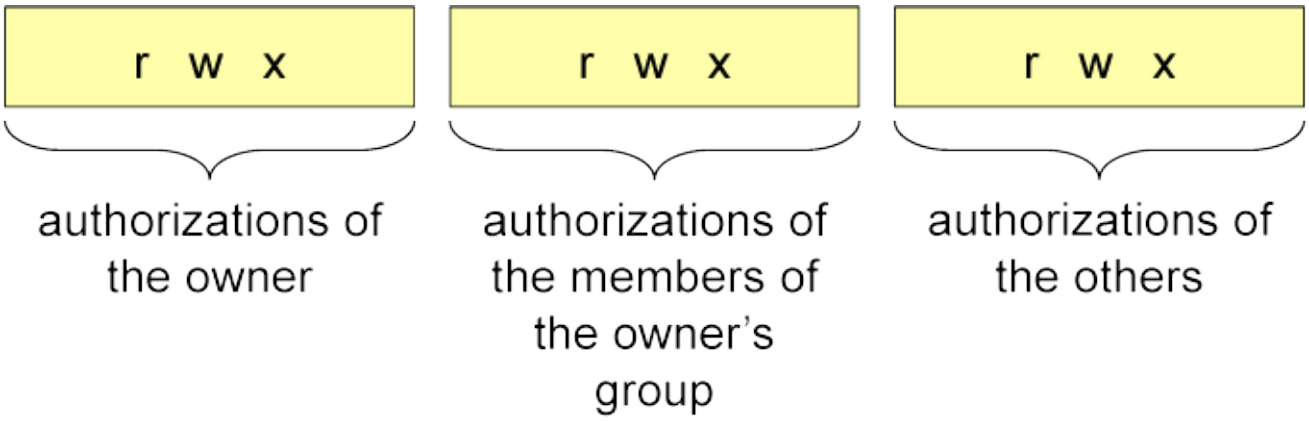


File descriptors

- File descriptors provide a handle to an inode.
- **fd = open(⟨NAME⟩, ⟨FLAGS⟩, ⟨MODE⟩)**
  - Retrieves the inode for file named ⟨**NAME**⟩.
  - Adds an entry **fd** $\mapsto$ **i-node** to the file descriptor table.
  - **fd** used afterwards to access the file, without name resolution.
- Example of flags and modes (permissions used if file is created)
  - **O_RDONLY**, **O_WRONLY**, **O_RDWR** for read only, write only, read/write.
  - **O_CREAT** creates a new file when the file does not exist.
  - if **O_EXCL** set along with **O_CREAT**, open fails if file exists.
  - **S_IRUSR** specifies owner read permission, if file is created.

Unix access control

- Access authorizations (**r**ead, **w**rite, and e**x**ecute), specified for user, group, and others.




```
-rw-r--r--   1 basin  users  6523 May 27 00:35 coding.tex
drwxr-xr-x   2 basin  users  2048 May 26 22:27 fig/
```

- For directories, **x** authorizes listing contents.
- Mode stored in inode attributes.

Static and dynamic links

- A link is a file f_1 that points to some other file f_2.

  f_2 can now be retrieved by naming f_1.

- A **static (hard) link** is a directory entry pointing directly to an inode.
  - **ln /bin/ls /bin/dir**
  - Name resolution for **/bin/dir** directly yields the inode.

- A **dynamic (symbolic) link** points to file containing target file's name.
  - **ln -s /bin/ls /bin/dir**
  - To retrieve inode, name resolution for **/bin/ls** is needed after the name resolution for **/bin/dir**.
  - One can make a symbolic link without access rights to linked file.

File name vulnerabilities

- Let's start with a simple problem: file names are not canonical.

  password, **/etc/password**, **../password** may all be the same.

- Due to links, directory tree is actually a graph, not a tree.

- File parsing vulnerabilities have been a problem in past.
  - Suppose FTP executes in **/pub** and should restrict access only to files in **/pub** and subdirectories.
  - Is it safe to allow only absolute addresses of form **/pub/...** or relative addresses?
  - No. Consider **../etc/password**.
  - A solution here is provided by Unix command **chroot**, which creates a "subdirectory jail".

- In general, parsing input can be tricky!

Unintended file modification

```
int my-file-append (char *text) {
   int fd;
   if ((fd = open ("/tmp/my-notes.tmp",
                   O_WRONLY | O_CREAT | O_APPEND,
                   S_IRUSR | S_IWUSR)                    ) <0 )
      return 1;
   if (write (fd, text) < 0) return 1;
   return 0:
}
```

- Opens file **/tmp/my-notes.tmp** for writing (**O_WRONLY**).
- Data appended to end of file (**O_APPEND**).
- File created if it doesn't exist (**O_CREAT**), whereby file's mode is **rw-------** (**S_IRUSR** | **S_IWUSR**).
- **/tmp/my-notes.tmp** intended to be a temporary file.
- Vulnerability? What happens if it is actually a symbolic link?

Unintended file modification (cont.)

```
int my-file-append (char *text) {
   int fd;
   if ((fd = open ("/tmp/my-notes.tmp",
                   O_WRONLY | O_CREAT | O_APPEND,
                   S_IRUSR | S_IWUSR)                    ) <0 )
      return 1;
   if (write (fd, text) < 0) return 1;
   return 0:
}
```

- Example attack
  - **rm /tmp/my-note.tmp**
  - **ln -s target-file /tmp/my-note.tmp**
  - If a user with rights to modify target-file calls this function, then text is appended to this file.
- A concrete instance: super-user executes function (or function is setuid to root) and target file is **/etc/password**.

A variant of previous problem

```
int my-file-append (char *text) {
   int fd;
   if ((fd = open ("/tmp/my-notes.tmp",
                   O_WRONLY | O_CREAT | O_APPEND,
                   S_IRUSR | S_IWUSR)                    ) <0 )
      return 1;
   if (write (fd, text) < 0) return 1;
   if (fchown (fd, getuid(), getgid() < 0) return 1;
   return 0:
}
```

- Might be employed in a setuid-root program so that caller owns the temporary file.
- Identical "linking attack" is possible.

  Result: user owns, e.g., **/etc/passwd**.

Recommendations

- Be aware that files may be links and check their status if needed before taking further actions.

- Status of file (link, owner, ...) can be determined by **lstat**.
  - **int lstat(char *filename, struct stat *stat)** returns **stat** structure on **filename**
  - For a symbolic link, returns status of file referenced by link.
  - Return value (0 or -1) indicates whether file exists or not.
  - **stat** structure provides other information such as file's owner and the file's type (regular or directory).

Applying the recommendations

```
int my-file-append (char *text) {
   int fd;
   struct stat stat;
   if (lstat("/tmp/my-notes.tmp", &stat) <0) {
     // file does not exist
     if ((fd = open ("/tmp/my-notes.tmp",
                     O_WRONLY | O_CREAT, S_IUSR | S_IWUSR)) <0 )
        return 1;
     if (fchown (fd, getuid(), getgid()) < 0) return 1;
   } else {
     // file already exists
     if (stat.st_uid != getuid()) return 1; // current user is not the owner
     if (!S_ISREG(stat.st_mode)) return 1; // file is not a regular file
     if ((fd = open ("/tmp/my-notes.tmp", O_WRONLY | O_APPEND)) < 0)
          return 1;
   }
   if (write (fd, text) < 0) return 1;
   return 0:
}
```

Does this program still have vulnerabilities?

Race conditions

- Race conditions occur when the results of computation depend on which thread or process is scheduled.
  - The result appears to be non-deterministic.
  - In reality, the result is determined by the scheduling algorithm and the environment.
- **Example:** suppose two threads append elements to a list.
  - Element ordering depends on when each thread is scheduled.
  - Race condition dangerous only if ordering is relevant.
- Race conditions are often unintended and difficult to identify.

Why is this so?

Race condition (1)

```
int my-file-append (char *text) {
  int fd;
  struct stat stat;
  if (lstat("/tmp/my-notes.tmp", &stat) <0) {
    // file does not exist
    if ((fd = open ("/tmp/my-notes.tmp",
                    O_WRONLY | O_CREAT, S_IUSR | S_IWUSR)) <0 )
      return 1;
    if (fchown (fd, getuid(), getgid()) < 0) return 1;
  } else {
    // file already exists
    if (stat.st_uid != getuid()) return 1; // current user is not the owner
    if (!S_ISREG(stat.st_mode)) return 1; // file is not a regular file
    if ((fd = open ("/tmp/my-notes.tmp", O_WRONLY | O_APPEND)) < 0)
        return 1;
  }
  if (write (fd, text) < 0) return 1;
  return 0:
}
```

- **lstat**
  - Performs name resolution to retrieve file's inode with associated status information
- **fd = open(...)**
  - Retrieves inode of file, again using name resolution.
  - Associates a file descriptor with file and opens file for writing.
- **fchown** changes owner and group of the file (in the inode).
- What is the problem (on failure branch)?

  Between **lstat** and **open**, attacker can symbolically link tmp-file to target file. Program changes target's owner.

Race condition (2)

```
int my-file-append (char *text) {
  int fd;
  struct stat stat;
  if (lstat("/tmp/my-notes.tmp", &stat) <0) {
    // file does not exist
    if ((fd = open ("/tmp/my-notes.tmp",
                    O_WRONLY | O_CREAT, S_IUSR | S_IWUSR)) <0 )
      return 1;
    if (fchown (fd, getuid(), getgid()) < 0) return 1;
  } else {
    // file already exists
    if (stat.st_uid != getuid()) return 1; // current user is not the owner
    if (!S_ISREG(stat.st_mode)) return 1; // file is not a regular file
    if ((fd = open ("/tmp/my-notes.tmp", O_WRONLY | O_APPEND)) < 0)
      return 1;
  }
  if (write (fd, text) < 0) return 1;
  return 0:
}
```

- **lstat**
  - Performs name resolution to retrieve file's inode with associated status information
- **if(!S_ISREG(stat.st_mode))** checks that file is regular i.e., is not a directory, block device, **symbolic link**, etc.
- **fd = open(...)**
  - Retrieves inode of file, again using name resolution.
  - Associates a file descriptor with file and opens file for writing.
- What is the problem (on success branch)?

  Between **S_ISREG** and **open** an attacker can replace file with a symbolic link to a target file. So program can overwrite target.

Recommendations

- Avoid race conditions!

  Ensure exclusive use of shared resources during critical command sequences, e.g., using locks or other mechanisms.

- For file systems, proceed as follows:

  1. **Open the file for access.**
  2. **Check the status of the file using the file descriptor.**
  3. **Access the contents of the file using the file descriptor.**

  In particular, do not use the file name in steps 2 and 3.
  I.e., **avoid repeated name resolution.**

- The problem of race conditions is not specific to C.

  E.g., occurs when using temporary files in shell scripts.

Employing the recommendations

```
int myappend(char *text) {
  int fd;
  struct stat st;

  if((fd=open ("/tmp/my-notes.tmp", O_WRONLY|O_APPEND|O_NOFOLLOW)) < 0) {
    if((fd=open ("/tmp/my-notes.tmp", O_WRONLY|O_EXCL|O_CREAT,
        S_IRUSR|S_IWUSR)) < 0)
            return 1;
    if(fchown(fd,getuid(),getgid())<0) return 1;
  }
  if(fstat(fd,&st) < 0) return 1;
  if(st.st_uid!=getuid()) return 1;
  if(!S_ISREG(st.st_mode)) return 1; // file could be a device
  if(st.st_nlink != 1) return 1;     // hard links
  if(write(fd,text)<0) return 1;
  return 0;
}
```

- Function uses **fstat** instead of **lstat**.
- **O_NOFOLLOW**: open fails if pathname is a symbolic link.
- **O_EXCL** stops creating a link between open calls.

  Recall, if **O_CREAT** and **O_EXCL** are set, open fails if file exists.
- inode used to check the existence of hard links.

Examples of past file-system vulnerabilities

- **lpr** offers the option to remove the file after printing.

  In early UNIX versions, anyone could print and remove the password file.

- **SETUID programs** (dynamic link)
  - Attacker creates a link "core" to the password file.
  - Attacker forces a core dump of a SETUID program.
  - The setuid program overwrites the password file.

- **mkdir** (dynamic link and race condition)
  - Early implementations first created an inode for the new directory and then changed ownership to the current user.
  - An attacker could remove the inode before ownership had been changed and create a dynamic link to the password file.
  - mkdir changes ownership of password file to the current user.

Conclusions and lessons learned

- Beware of programs that mix data and control.
- In C, variable number of arguments are dangerous.
- **Printf** is more powerful than you thought.
- Names, and in particular symbolic ones, cause problems.
- Race conditions can be quite subtle.

  Eliminating them requires detailed understanding of which operations are atomic.